IE University

School of Science & Technology

Hierarchical N-Body Simulation of Galactic Dynamics in WebGPU

Enabling Scalable and Interactive Physics Simulations on Modern Web Platforms

Zaid Alsaheb

Bachelor of Computer Science & Artificial Intelligence

Supervised by: Professor Raul Perez Pelaez

1) Literature Review

Accurate simulation of galactic dynamics requires resolving gravitational interactions across large numbers of particles while maintaining numerical stability over long integration times. Achieving this balance has historically required substantial computational resources and specialized software, placing high-fidelity N-body simulations largely outside the reach of lightweight or widely accessible platforms. As a result, the relevant literature spans several traditionally separate domains, including astrophysical N-body methods, numerical integration techniques, GPU-accelerated computing, and browser-based visualization technologies.

This literature review is organized to clarify how these areas intersect and to highlight the technical gap addressed by this work. It first examines the physical and algorithmic foundations of gravitational N-body simulation, with particular attention to the computational scaling challenges that motivate hierarchical approximation methods such as Barnes–Hut. It then surveys prior efforts to accelerate N-body simulations using GPU architectures, with an emphasis on new emergent technologies that allow to achieve native performance in web browsers, drastically enhancing the range of devices that can perform these simulations.

Computational Constraints in Astrophysical Simulation

Over the past decades, progress in computational astrophysics has closely followed the performance improvements predicted by Moore's Law, with central processing unit (CPU) speeds increasing at a near-exponential rate. Once an algorithm was implemented, substantial performance gains could often be achieved simply by running existing code on newer hardware, with minimal additional development effort. However, as single-core CPU performance has plateaued (Sutter, 2005), this implicit scaling model has begun to break down. As a result, continued advances in computational astrophysics increasingly depend on exploiting parallelism and adapting algorithms to emerging computing architectures (Fluke et al., 2011).

One of the most significant architectural shifts has been the widespread adoption of graphics processing units (GPUs), which offer massive parallelism and high memory bandwidth. While GPUs provide the potential for orders-of-magnitude performance improvements (Owens et al., 2007), realizing these gains typically requires substantial algorithmic reformulation. Algorithms that scale well on serial or modestly parallel CPUs may perform poorly on highly parallel architectures if they involve irregular control flow or memory access pattern (Burtscher & Pingali, 2011).

Gravitational N-Body Simulations and Quadratic Scaling

Among the most computationally demanding problems in astrophysics are gravitational N-body simulations, which form the foundation of many studies in galactic dynamics and structure

formation (Binney & Tremaine, 2008). In such simulations, a galaxy is modeled as a system of particles (representing stars or dark matter) whose evolution is governed by their mutual gravitational interactions (Zwart et al., 2007).

According to Newton's law of universal gravitation, the force exerted on a particle of mass m_i by another particle of mass m_j is given by

$$\vec{F}_{ij} = G\frac{m_i m_j(\vec{r}_j - \vec{r}_i)}{|\vec{r}_j - \vec{r}_i|^3} \tag{1}$$

Where $\vec{F}_{ij}$ denotes the gravitational force exerted on particle i by particle j, and G is the gravitational constant. The quantities m_i and m_j represent the masses of particles i and j, respectively. The vectors $\vec{r}_i$ and $\vec{r}_j$ give the positions of particles i and j in three-dimensional space.

The vector difference $\vec{r}_j - \vec{r}_i$ points from particle i toward particle j, while $|\vec{r}_j - \vec{r}_i|$ denotes the Euclidean distance between the two particles. The cubic power of the distance in the denominator ensures that the magnitude of the force follows an inverse-square law while preserving the correct force direction.

The total gravitational force acting on particle i is obtained by summing the pairwise force contributions from all other particles in the system, excluding self-interaction,

$$\vec{F}_i = \sum_{j \neq i}^{N} G\frac{m_i m_j(\vec{r}_j - \vec{r}_i)}{|\vec{r}_j - \vec{r}_i|^3} \tag{2}$$

Evaluating this expression requires computing $N-1$ pairwise interactions per particle. Consequently, a direct implementation of gravitational force evaluation scales as $O(N^2)$ per simulation timestep. This quadratic scaling rapidly becomes computationally prohibitive as the number of particles increases, particularly when long integration times are required to observe large-scale dynamical phenomena such as spiral structure or halo evolution (Zwart et al., 2007).

Hierarchical Approximation and the Barnes–Hut Algorithm

To address the computational limitations imposed by direct force evaluation, a variety of approximation techniques have been developed to reduce the cost of N-body simulations while preserving essential physical behavior. One of the most influential of these methods is the Barnes–Hut algorithm, which exploits the hierarchical spatial structure of particle distributions to approximate gravitational interactions (Barnes & Hut, 1986).

Originally introduced by Barnes and Hut in 1986, the algorithm organizes particles into a tree structure, typically a quadtree in two dimensions or an octree in three dimensions. During force evaluation, distant groups of particles are approximated as a single effective mass located

at their center of mass, allowing subsets of the force summation to be replaced by a single interaction. An opening-angle parameter controls the accuracy of this approximation, providing a tunable tradeoff between computational efficiency and force accuracy.

By replacing many distant pairwise interactions with aggregate approximations, Barnes–Hut reduces the computational complexity of force evaluation from $O(N^2)$ to approximately $O(N \log N)$. This reduction makes the algorithm particularly well-suited for simulating large, collisionless systems such as galaxies, where global dynamical behavior is often of greater interest than exact short-range interactions.

Numerical Integration and Stability Considerations

In addition to force evaluation, the accuracy and long-term stability of N-body simulations depend critically on the numerical integration of the equations of motion. In gravitational N-body problems, the primary quantity of interest is the time evolution of the particle positions, which is governed by Newton's equations of motion. For each particle i, the total gravitational force resulting from interactions with all other particles determines its acceleration according to

$$\sum_j \vec{F}_{ij} = m_i \vec{a}_i \tag{3}$$

Where $\vec{a}_i$ is the acceleration of the particle i. The acceleration is related to the particle velocity and position through time derivatives,

$$\vec{a}_i = \frac{d\vec{v}_i}{dt}, \qquad \vec{v}_i = \frac{d\vec{r}_i}{dt} \tag{4}$$

Because these equations generally cannot be solved analytically for systems containing many interacting particles, their evolution must be approximated numerically. Numerical integration methods advance the particle positions and velocities forward in time using discrete timesteps of size Δt, approximating the continuous dynamics of the system.

One of the simplest numerical integration schemes is the forward Euler method (Kreyszig, 2011). In this approach, particle positions are updated using the current velocity according to

$$\vec{r}_i^{\,n+1} = \vec{r}_i^{\,n} + \vec{v}_i^{\,n} \Delta t \tag{5}$$

Where the superscript n denotes the discrete timestep, with $\vec{r}_i^{\,0}$ representing the initial particle position. Velocities are updated similarly using the acceleration computed from the forces at the current timestep. While the Euler method is computationally inexpensive and straightforward to implement, it suffers from poor numerical stability and does not conserve energy, leading to significant accumulated errors and unphysical behavior over long integration times.

As a result, astrophysical N-body simulations commonly employ symplectic integration schemes such as the Verlet or leapfrog methods, which are specifically designed to preserve the Hamiltonian structure of the equations of motion. These methods exhibit significantly improved long-term energy conservation and are therefore better suited for simulations of galactic dynamics that require stable evolution over many dynamical timescales (Springel, 2005).

In Barnes–Hut simulations, errors introduced by numerical integration interact with approximation errors arising from hierarchical force evaluation (Salmon & Warren, 1994). Prior studies have shown that, for many galactic-scale simulations, integration error can dominate force-approximation error when inappropriate timestepping schemes or integration methods are used (Springel, 2005). Consequently, careful selection of numerical integration techniques remains essential even when approximate force models are employed.

GPU Acceleration of Hierarchical N-Body Methods

The computational demands of gravitational N-body simulations have made them a natural target for graphics processing units (GPUs), which provide massive data parallelism and high memory bandwidth. Modern GPUs are designed with thousands of small, efficient cores capable of executing the same operation on many data elements simultaneously (Fluke et al., 2011). This architecture is particularly well-suited to the independent, particle-wise force computations inherent in N-body simulations (Nyland et al., 2009).

To leverage GPUs effectively, computational tasks are expressed in terms of small programs that run on the GPU cores called shaders. Originally, shaders were designed to compute visual effects for rendering pipelines, including vertex transformations, fragment coloring, and texture operations. Early GPU-based scientific computing therefore relied on repurposing graphics shaders for numerical tasks, encoding computation as rendering operations and storing data in textures (Owens et al., 2008).

Over time, GPU programming languages such as CUDA and OpenCL have generalized this model to allow general-purpose computation, enabling the parallel execution of tasks that are not directly related to graphics, including N-body simulations (Fluke et al., 2011; Nyland et al., 2009).

Despite these advances, hierarchical algorithms such as Barnes–Hut present persistent challenges for GPU architectures. Tree construction and traversal introduce irregular memory access patterns and branch divergence, both of which reduce parallel efficiency on SIMD-style hardware (Burtscher & Pingali, 2011; Karras, 2012). To mitigate these issues, prior work has proposed optimizations including linearized tree representations, stackless traversal methods, and the use of space-filling curves to improve memory coherence (Burtscher & Pingali, 2011;

Gaburov et al., 2010). While these techniques have enabled efficient native GPU implementations, they rely on low-level memory management and flexible data structures that have traditionally been inaccessible in browser-based computing environments.

Since the introduction of the Barnes–Hut algorithm, numerous modifications and alternative hierarchical methods have been proposed to improve accuracy and efficiency, including higher-order multipole expansions and the Fast Multipole Method (FMM) (LF & Rokhlin, 2001; Nyland et al., 2009; Wang, 2021). These approaches differ primarily in how distant particle interactions are approximated and aggregated. Despite these developments, Barnes–Hut remains widely used due to its favorable balance between computational cost, accuracy, and implementation complexity.

General Purpose GPU Computation in the Web

Modern web applications increasingly rely on GPU acceleration to deliver interactive, intelligent, and computation-heavy experiences directly in the browser. Access to GPU resources enables use cases such as real-time large-model reasoning, embedding generation, multimedia processing, and advanced data visualization without requiring users to install specialized software or manage local hardware (Congote et al., 2011; Sung et al., 2025; Usta, 2024). For users working across devices or within managed environments e.g., students, researchers, enterprise teams), browser-based GPU access lowers friction, improves accessibility, and supports scalable, on-demand compute.

WebGL has long served as the primary mechanism for accessing GPU acceleration within web browsers. Standardized by the Khronos Group as a JavaScript binding to OpenGL ES, WebGL was designed to provide portable and efficient access to GPU hardware for real-time graphics rendering across a wide range of devices (Khronos Group, 2024). Its success has enabled a broad ecosystem of browser-based visualization tools, interactive simulations, and educational applications.

Although WebGL exposes programmable shaders, its computational model is fundamentally centered around the graphics rendering pipeline. All computation must be framed in terms of vertex and fragment processing stages, with data represented as textures and intermediate results captured via framebuffers (Congote et al., 2011). As a consequence, general-purpose computation in WebGL requires reformulating numerical algorithms as sequences of rendering passes, often involving non-intuitive data layouts and multiple shader invocations to emulate iteration and state updates.

This graphics-centric abstraction has enabled a class of data-parallel algorithms to be executed efficiently in the browser, particularly those that map naturally onto image-based

representations. However, it imposes significant constraints on algorithms that require irregular memory access, dynamic data structures, or complex control flow (Congote et al., 2011) . In the context of gravitational N-body simulations, these constraints complicate the implementation of adaptive time stepping, hierarchical spatial decomposition, and recursive tree traversal, all of which are central to physically accurate Barnes–Hut simulations (Burtscher & Pingali, 2011).

As a result, prior WebGL-based N-body implementations have typically prioritized real-time performance and visual plausibility over numerical rigor. Common approaches include limiting particle counts, employing softened or approximate force models, reducing integration accuracy, or relying on fixed spatial grids rather than fully adaptive hierarchical trees (Sengupta et al., 2025). While such methods are effective for interactive visualization and outreach, they are not well suited to large-scale galactic simulations where long-term energy conservation and accurate force evaluation are essential.

WebGPU is a modern web standard designed to provide low-level, high-performance access to GPU hardware from within the browser. Unlike WebGL, which is built around a traditional graphics pipeline, WebGPU exposes explicit support for general-purpose GPU computation through compute shaders, storage buffers, and programmable pipelines (GPU for the Web Working Group, 2026). Its design closely mirrors contemporary native graphics and compute APIs such as Vulkan, Metal, and Direct3D 12, enabling fine-grained control over memory layout, synchronization, and parallel execution (Usta, 2024).

Central to WebGPU's computational model is the compute shader: a general-purpose GPU program that executes independently of the rendering pipeline. Compute shaders allow developers to express parallel workloads directly in terms of data processing rather than image generation, making them well suited to scientific computing tasks such as particle simulation, spatial partitioning, and force evaluation (Sung et al., 2025; Usta, 2024). This model enables algorithms to be structured in a manner similar to native GPU implementations, without the need for graphics-specific workarounds (Sengupta et al., 2025).

The explicit memory model and flexible buffer abstractions provided by WebGPU make it possible to represent complex data structures, including hierarchical trees and linearized spatial indices, directly on the GPU (Sung et al., 2025; Usta, 2024). These features are particularly relevant for Barnes–Hut simulations, which rely on dynamic tree construction and traversal to achieve sub-quadratic scaling. While such patterns were previously impractical in web environments, WebGPU enables their implementation with performance characteristics comparable to native GPU code, subject to browser and hardware constraints (Sengupta et al., 2025).

2) Methodology

2.1 Research Design and Objectives

This work adopts a computational-methods design in which a gravitational N-body solver is implemented and evaluated with a focus on reproducibility, long-term numerical stability, and computational scalability beyond the $O(N^2)$ cost of direct summation. The solver is implemented in C++20 using the WebGPU C API and built from a single codebase targeting both native desktop execution (via backends such as wgpu-native and Dawn) and browser execution (compiled via Emscripten) (Khronos Group, 2024). This dual-target approach enables interactive visualization as well as headless batch runs, allowing performance and numerical behavior to be evaluated under comparable conditions across platforms.

The physical and numerical foundations of the implementation draw directly from the literature on hierarchical N-body simulation and GPU parallelism. The gravitational force model adopts Newtonian gravity with Plummer-type softening to avoid the $\frac{1}{r^2}$ singularity and reduce spurious two-body relaxation in collisionless regimes (Binney & Tremaine, 2008). Accelerations are computed using a Barnes–Hut-style hierarchical approximation that reduces force evaluation from $O(N^2)$ to approximately $O(N \log N)$ (Barnes & Hut, 1986), with the primary code path constructing and traversing a Linear Bounding Volume Hierarchy (LBVH) entirely on the GPU using the parallel method of Karras (Karras, 2012). Time integration employs a second-order symplectic leapfrog scheme (kick–drift–kick), selected for its improved long-term energy behavior relative to forward Euler in gravitational systems (Springel, 2005). The compute platform, WebGPU, provides general-purpose parallel compute shaders that enable both native and browser deployment through the same low-level API surface.

The evaluation is structured around three operational research questions:

1. **Scalability**: How does runtime per timestep scale with N for a WebGPU Barnes–Hut implementation compared to a direct $O(N^2)$ baseline at small N?
2. **Numerical quality**: For fixed N, how do timestep size Δt and opening angle θ affect (a) long-term conservation behavior (energy and momentum drift where measurable) and (b) stability under long integrations?
3. **Platform feasibility**: What particle counts and timestep rates are practical under WebGPU constraints (32-bit GPU arithmetic, buffer and memory limits, scheduling overhead, and device variability)?

The choice of WebGPU as the compute platform is central to all three research questions; the following section provides a detailed justification.

2.2 WebGPU Platform Justification

The Barnes–Hut algorithm at $O(N \log N)$ requires a full tree rebuild and traversal at every timestep, each involving operations over the entire particle set. At the particle counts targeted by this work (N up to 10^5), these operations are dominated by data-parallel phases: computing bounding boxes, sorting Morton codes, constructing tree topology, and evaluating forces across all particles simultaneously. This degree of parallelism makes GPU execution essential for interactive frame rates, a conclusion broadly supported by the literature on GPU-accelerated N-body simulation (Fluke et al., 2011; Owens et al., 2007; 2008).

The specific choice of WebGPU over established GPU APIs such as CUDA and OpenCL is motivated by three factors. First, WebGPU is a cross-platform compute and rendering API that exposes general-purpose compute shaders and storage buffers through a unified interface, running natively on Vulkan, Metal, and Direct3D 12 backends while also being deployable in web browsers (GPU for the Web Working Group, 2026; Khronos Group, 2024). This portability is central to the research objective of evaluating N-body simulation across diverse execution environments, from native desktop applications to browser-based deployments. Second, WebGPU's compute shader model provides the primitives required for tree-based algorithms: random-access storage buffer reads and writes, atomic operations for bottom-up aggregation, and flexible workgroup dispatch (Sung et al., 2025; Usta, 2024). Third, WebGPU is an emerging W3C standard with active implementation across major browsers and native runtimes, positioning it as the successor to WebGL for GPU-accelerated web applications (Sengupta et al., 2025).

These advantages are accompanied by constraints that inform the implementation design. GPU-side computation in WebGPU is limited to 32-bit floating-point arithmetic, imposing a precision ceiling on force evaluation and integration. Buffer size limits and memory allocation patterns vary across devices and backends. Scheduling overhead for compute dispatches can be significant relative to kernel execution time at small N, and device variability across integrated and discrete GPUs affects both performance and available features (Sengupta et al., 2025). These constraints are explicitly tested through the platform-feasibility research question.

A comparative assessment of available GPU APIs clarifies the positioning of WebGPU. CUDA provides mature tooling and high performance for tree-based N-body codes (Burtscher & Pingali, 2011; Gaburov et al., 2010), but is locked to NVIDIA hardware and has no browser deployment path. OpenCL offers cross-vendor support on desktop but lacks browser integration and has seen declining adoption in favour of vendor-specific APIs. WebGL provides broad browser reach but was designed for rendering, not general-purpose computation: it

lacks compute shaders, storage buffers, and atomic operations, limiting it to fragment-shader workarounds for GPGPU tasks (Congote et al., 2011). WebGPU is uniquely positioned as both browser-deployable and compute-capable, making it the only viable API for evaluating an N-body solver across native and web environments from a single codebase.

Criterion	**CUDA**	**OpenCL**	**WebGL**	**WebGPU**
Compute shaders	Yes	Yes	No	Yes
Storage buffers	Yes	Yes	No	Yes
Atomic operations	Yes	Yes	No	Yes
Browser deployment	No	No	Yes	Yes
Cross-vendor GPU	No	Yes	Yes	Yes
Native deployment	Yes	Yes	No	Yes
64-bit GPU float	Yes	Varies	No	No

Table 1: Comparison of GPU compute APIs against criteria required for hierarchical N-body simulation. WebGPU is the only API that combines compute shader support with browser deployability.

With the computational platform established, the following sections describe the physical model and numerical methods that operate within it.

2.3 Physical Model and State Representation

2.3.1 Softened gravitational acceleration

Each particle represents a mass element evolving under self-gravity in an isolated (open) domain. Using dimensionless units with $G = 1$, the softened acceleration of particle i is

$$a_i = G \sum_{j \neq i} m_j \frac{r_j - r_i}{\left(\|r_j - r_i\|^2 + \varepsilon^2\right)^{\frac{3}{2}}} \tag{6}$$

The softening parameter ε (default 0.5, configurable) introduces a Plummer-type potential that suppresses the $\frac{1}{r^2}$ singularity at short range (Binney & Tremaine, 2008). This formulation is equivalent to treating each particle as a Plummer sphere rather than a point mass, and the same softening kernel appears in the Plummer-model initial conditions used in Scenario B (see Section 2.9).

2.3.2 GPU-friendly mass packing

To reduce memory bandwidth on the GPU, each particle's mass is stored in the w component of its position vector, yielding a packed `vec4<f32>` layout of (x, y, z, m) that avoids a dedicated mass buffer. Because GPU compute throughput is frequently limited by memory bandwidth rather than arithmetic capacity, this packing halves the number of buffer reads required during force evaluation compared to a separate position and mass layout.

Component	.x	.y	.z	.w
positions[i]	x_i	y_i	z_i	m_i
velocities[i]	$v_{x,i}$	$v_{y,i}$	$v_{z,i}$	0
accelerations[i]	$a_{x,i}$	$a_{y,i}$	$a_{z,i}$	0

Table 2: Particle state layout using vec4<f32> packing. Mass is stored in the w component of the position vector to reduce buffer count and memory bandwidth during force evaluation.

2.3.3 Precision strategy

GPU kernels operate in 32-bit floating point to maximize throughput and match typical WebGPU device capabilities. Diagnostic quantities (total energy and momentum) are computed on the CPU in double precision to reduce accumulation error in long-running integrations. This split reflects the practical precision-versus-performance trade-off inherent in WebGPU environments, where 64-bit GPU arithmetic is not available (Sengupta et al., 2025).

Given the acceleration field defined above, the system state is advanced in time using the integration scheme described in the following section.

2.4 Time Integration

2.4.1 Primary integrator: symplectic leapfrog (KDK)

The primary integration scheme is a fixed-timestep, second-order symplectic leapfrog (kick–drift–kick) (Verlet, 1967). With timestep Δt (default 0.0001), the update sequence is:

1. Half-kick:

$$v_i^{n+1/2} = v_i^n + \frac{\Delta t}{2} a_i^n \tag{7}$$

2. Drift:

$$r_i^{n+1} = r_i^n + \Delta t v_i^{n+1/2} \tag{8}$$

3. Recompute acceleration a_i^{n+1} using updated positions.
4. Half-kick:

$$v_i^{n+1} = v_i^{n+1/2} + \frac{\Delta t}{2} a_i^{n+1} \tag{9}$$

This choice is motivated by the well-known long-term stability advantages of symplectic schemes in gravitational dynamics (Springel, 2005). Unlike forward Euler, which introduces secular energy drift proportional to Δt, the leapfrog scheme is time-reversible and exhibits bounded energy oscillation rather than monotonic growth, making it substantially more suitable for long-horizon integrations in Hamiltonian systems (Binney & Tremaine, 2008). The energy

conservation behavior of leapfrog relative to Euler is quantified in the evaluation protocol (see Section 2.10).

2.4.2 Euler integrator (baseline and fallback)

A forward Euler method is retained as `--integrator euler` to provide a stability baseline. Its update sequence is: tree build, force evaluation, then

$$v \leftarrow v + a\Delta t, \;\; r \leftarrow r + v\Delta t \tag{10}$$

The Euler integrator serves as a lower bound on numerical quality against which the leapfrog scheme is compared.

The most expensive step in each integration cycle is the evaluation of accelerations, addressed through hierarchical approximation in the following section.

2.5 Hierarchical Force Evaluation

2.5.1 Monopole approximation

Hierarchical evaluation approximates distant particle groups by a single monopole at the node's center of mass. For a node with total mass M and center of mass R,

$$a_{i,\text{node}} = GM \frac{R - r_i}{(\|R - r_i\|^2 + \varepsilon^2)^{\frac{3}{2}}} \tag{11}$$

This monopole approximation is used consistently across both tree topologies implemented in this work: a binary Bounding Volume Hierarchy (BVH) on the GPU and an eight-way octree on the CPU. Only the tree representation and the geometry of the opening criterion differ between the two paths.

2.5.2 Opening criteria

An internal node is accepted as a monopole approximation when it is sufficiently small relative to its distance from the target particle. The GPU BVH path uses a tight axis-aligned bounding box (AABB) and tests whether the squared maximum extent of the node satisfies $\frac{\text{maxExtent}^2}{d^2} < \theta^2$, where $\text{maxExtent} = \max(\Delta x, \Delta y, \Delta z)$ is derived from the node's AABB bounds. This extent-based criterion reflects the non-cubic node shapes that arise in a BVH more accurately than a uniform half-width. The CPU octree path uses the conventional half-width criterion $\frac{h^2}{d^2} < \theta^2$, where h is the half-width of the cubic octree cell and $d^2 = \|r_i - R\|^2$. Both formulations are expressed in squared form to avoid a per-node square root. Salmon and Warren provide rigorous error bounds for these opening criteria, showing that the conventional Barnes–Hut criterion can admit unbounded errors for $\theta \geq 1/3$ in pathological configurations (Salmon & Warren, 1994).

The default opening angle $\theta = 0.75$ is used as a practical balance between accuracy and performance. For comparison, GADGET-2 uses θ values in the range 0.5 to 0.7 for cosmological

simulations where higher force accuracy is required (Springel, 2005), while the original Barnes and Hut paper used $\theta = 1.0$ (Barnes & Hut, 1986). The parameter sweeps in the evaluation protocol (see Section 2.10) systematically characterize the accuracy–performance trade-off across $\theta \in \{0.3, 0.5, 0.7, 1.0\}$.

Placeholder: Tree opening criterion geometry showing a target particle, a distant tree node, the distance d, the node extent, and the θ threshold for both BVH (maximum AABB extent) and octree (half-width) variants.

Figure 1: Geometry of the opening criterion. A node is approximated as a monopole when its angular size, as measured by extent/d, falls below the threshold θ. Left: BVH variant using maximum AABB extent. Right: octree variant using cell half-width.

The dual-topology approach motivates a software architecture with multiple execution paths, described in the following section.

2.6 Software Architecture and Execution Modes

The implementation is written in C++20 for host-side orchestration and physics, with WGSL for compute and rendering shaders, using the WebGPU C API directly without wrapper libraries. The build system uses CMake with FetchContent and pinned dependency versions to ensure deterministic builds. Dependencies fetched automatically include WebGPU-distribution (v0.2.0), GLFW (3.4), glfw3webgpu (v1.2.0), spdlog (v1.16.0), Dear ImGui (v1.90.9), and GLM (1.0.2). Three build backends are supported: `WGPU` (wgpu-native), `DAWN` (Dawn), and `EMSCRIPTEN` (browser build with `-sASYNCIFY`, `-sALLOW_MEMORY_GROWTH=1`, `-sUSE_GLFW=3`).

The primary execution mode is GPU-primary: all physics operations (integration, tree construction, and force evaluation) are performed on the GPU each step. Diagnostic quantities are computed on the CPU only at configurable intervals through staging-buffer readback of positions and velocities, with the diagnostic frequency set to every 60 frames in interactive mode and every step or every 50 steps in headless mode depending on N. To support cross-checking and fallback behavior, CPU mirror arrays (`cpuPositions_`, `cpuVelocities_`, `cpuAccelerations_`) are retained and used by non-primary modes (Euler and CPU-tree leapfrog), where scalar CPU loops and a CPU octree are executed in parallel with the GPU path. This design yields two

methodological advantages: first, no per-step CPU physics overhead in the primary mode, since the tree is built directly from GPU-resident particle state; second, cross-validation potential across runs, since selecting the CPU octree versus GPU BVH path provides two independent hierarchical implementations for comparative diagnostics.

Placeholder: System architecture block diagram showing C++ host orchestration, WebGPU compute shader dispatch, GPU storage buffers (positions, velocities, accelerations, BVH nodes), staging buffer readback for diagnostics, CPU mirror arrays for fallback paths, and the rendering pipeline reading directly from storage buffers.

Figure 2: System architecture overview. The GPU-primary path (solid arrows) performs all physics on-device. CPU mirror arrays (dashed arrows) support fallback execution modes and cross-validation. Diagnostic readback occurs at configurable intervals via staging buffers.

The GPU-primary execution mode relies on a set of WebGPU compute shaders organized into a per-timestep pipeline, described in the following section.

2.7 WebGPU Compute Methodology

2.7.1 GPU data layout

Simulation state is stored in WebGPU storage buffers using the packed `vec4<f32>` layout described in Table 2: positions (with mass in the w component), velocities, and accelerations. The BVH is stored as an array of `BVHNode` structures with $2N - 1$ entries (internal nodes indexed 0 to $N - 2$, leaves indexed $N - 1$ to $2N - 2$), where each node stores a center of mass and a tight AABB. A flattened CPU octree buffer is maintained for fallback paths, using explicit child fields (`c0` through `c7`) to avoid dynamic indexing limitations in WGSL. A uniform parameters buffer shares layout-matched simulation parameters between C++ and WGSL. In-place updates are used throughout: there is no double buffering of positions, velocities, or accelerations. Correct ordering between compute passes relies on WebGPU's implicit storage-buffer synchronization between dispatches within a single command buffer submission, with bind groups recreated each frame.

2.7.2 Compute shaders

The implementation comprises twelve WGSL compute shaders organized into two functional groups. The first group handles integration and force evaluation: a direct-summation baseline shader, an octree-traversal fallback shader, the primary BVH-traversal force shader, and kick/drift and Euler integration shaders. The second group implements LBVH construction in seven passes: two-pass bounding-box reduction, Morton code generation (Morton, 1966), bitonic sort with multiple sub-passes (Batcher, 1968), Karras topology construction (Karras, 2012), leaf initialization, and bottom-up aggregation via atomic counters. Workgroup sizes are fixed per kernel (64 for force evaluation, 256 for integration and tree building) and are reported as part of the implementation configuration.

2.7.3 Per-timestep execution sequence (GPU-primary leapfrog)

Each timestep is recorded into a single command encoder and submitted as one command buffer. The six sequential passes are:

1. Half-kick: $v \leftarrow v + \frac{a\Delta t}{2}$
2. Drift: $r \leftarrow r + v\Delta t$
3. LBVH build (7 sub-passes): global AABB reduction → Morton code generation → bitonic sort → Karras topology construction → leaf initialization → bottom-up aggregation
4. BVH force evaluation: iterative traversal with fixed-depth explicit stack (depth 64, sufficient for all tested N)
5. Half-kick: $v \leftarrow v + \frac{a\Delta t}{2}$
6. Diagnostics readback (periodic): stage-map-readback → CPU double-precision energy and momentum computation

Placeholder: Vertical flow diagram showing the six per-timestep passes: half-kick, drift, LBVH build (with 7 sub-passes expanded), BVH force evaluation, half-kick, and periodic diagnostics readback.

Figure 3: Per-timestep GPU execution pipeline for the leapfrog integrator. All six passes are recorded into a single command buffer. The LBVH build (pass 3) comprises seven sub-passes with implicit barrier synchronization between each.

2.7.4 GPU traversal (iterative, no recursion)

Tree traversal is implemented iteratively in the BVH force shader. One GPU thread is assigned per particle. The thread maintains an explicit stack of node indices, beginning from the root. A node is either accepted (leaf or opening criterion satisfied) and accumulated via the monopole approximation, or expanded by pushing its children onto the stack. Fast inverse square root (`inverseSqrt`) is used for inverse-distance evaluation, and a self-interaction guard avoids contributions from degenerate near-zero distances. This approach preserves the Barnes–Hut approximation structure while accommodating GPU execution constraints and limiting branch divergence where possible (Burtscher & Pingali, 2011; Karras, 2012; Nyland et al., 2009).

2.7.5 GPU LBVH construction (Karras 2012)

The LBVH is built fully on-device in seven conceptual steps. First, a two-pass parallel reduction computes the global axis-aligned bounding box. Second, particle positions are normalized to a $[0, 1023]^3$ integer grid and their bits interleaved to produce 30-bit Morton codes (Morton, 1966), which impose a spatial ordering along a space-filling Z-curve. Third, Morton codes and associated particle indices are sorted using a bitonic sort network (Batcher, 1968) with multiple (k, j) sub-passes and dynamic uniform offsets; arrays are padded to the next power of two, with sentinel codes assigned to padding elements. Fourth, the parallel binary tree topology is constructed using the Karras delta function (leading zeros of the XOR of adjacent codes, with tie-breaking for duplicates) (Karras, 2012). Fifth, leaves are initialized by mapping sorted indices to particle positions, masses, and point AABBs. Sixth, internal-node AABBs and

centers of mass are aggregated bottom-up using atomic counters to ensure both children are processed before their parent. The resulting BVH is immediately traversable without CPU-side construction or upload, eliminating a per-step CPU bottleneck in the primary execution mode.

Placeholder: Vertical flow diagram showing the seven LBVH construction sub-passes with data dependencies: (1) global AABB reduction, (2) Morton code generation, (3) bitonic sort, (4) Karras topology, (5) leaf initialization, (6) bottom-up aggregation. Arrows indicate barrier synchronization points between passes.

Figure 4: LBVH construction pipeline. Seven compute dispatches build the tree entirely on-device. Each arrow represents an implicit storage-buffer barrier. The atomic-counter aggregation in pass 6 ensures correct bottom-up propagation of bounding boxes and centers of mass.

2.7.6 CPU octree construction (fallback paths)

The CPU octree is used only by Euler and CPU-tree leapfrog modes. It is built from CPU mirror arrays by computing a bounding box, inserting particles via octant selection, propagating centers of mass bottom-up, and optionally flattening to a GPU-friendly node array when GPU evaluation is used. GPU buffers auto-resize during uploads when needed.

In interactive mode, the GPU-computed particle state is rendered directly from storage buffers, as described in the following section.

2.8 Rendering and Interactive Operation

For visualization, particles are rendered as instanced billboard quads with additive blending. Positions and colors are read directly from storage buffers via `@builtin(instance_index)`, requiring no separate vertex buffer. Depth testing is enabled with depth writes disabled; fragments are masked to a circular footprint with a soft alpha falloff. An ImGui overlay provides interactive control of simulation parameters (θ, Δt, ε, and integrator selection) and displays real-time diagnostics including energy, momentum, and per-component timing breakdowns. In headless mode, the rendering pipeline is skipped entirely and the simulation loop runs without

any presentation overhead, enabling pure throughput measurement. Rendering is a presentation layer and does not alter the simulation state.

Having described the solver and its implementation, the following section specifies the benchmark scenarios used to evaluate it.

2.9 Initial Conditions and Benchmark Scenarios

No external astronomical datasets are used. All experiments are generated from synthetic initial conditions and produce derived outputs (trajectories, diagnostic scalars, and timing logs). This design eliminates licensing and privacy concerns and enables controlled, repeatable comparisons across parameter sweeps. Each experiment is fully specified by command-line parameters: scenario type, seed, N, Δt, θ, softening ε, and step count.

2.9.1 Scenario A: two-body orbit (sanity check)

Scenario A places two equal-mass particles ($m = 1000$ each, $N = 2$ enforced regardless of the `--N` parameter) separated by $d = 10$ units along the x-axis, with tangential velocities along the z-axis computed for a softened circular orbit:

$$v = \sqrt{\frac{Gmd^2}{2(d^2 + \varepsilon^2)^{\frac{3}{2}}}} \tag{12}$$

This configuration provides the simplest possible validation of integrator correctness. With the correct timestep and softening, the two particles should maintain a stable circular orbit indefinitely under the leapfrog scheme. Departures from circularity provide a direct diagnostic for integration error, and the sensitivity to Δt and ε can be isolated without confounding effects from hierarchical force approximation. The scenario is not representative of large-N hierarchical behavior, but it is an essential sanity check that must pass before more complex scenarios are credible.

2.9.2 Scenario B: Plummer sphere (spherical equilibrium test)

Scenario B generates a Plummer sphere (Aarseth et al., 1974; Plummer, 1911) with $N \in [10^3, 10^5]$ (depending on hardware) and scale length $a = 5$. The Plummer model is a classical self-gravitating equilibrium configuration in which the density profile follows $\rho(r) \propto \left(1 + \frac{r^2}{a^2}\right)^{-\frac{5}{2}}$, and it is widely used as a test case for N-body codes because of its known analytic properties (Binney & Tremaine, 2008). Particle radii are sampled via the inverse cumulative distribution function

$$r = \frac{a}{\sqrt{u^{-2/3} - 1}} \tag{13}$$

with u clamped to $[0.001, 0.999]$. Angular coordinates are isotropic: $\cos(\theta)$ is drawn uniformly and the azimuthal angle ϕ is drawn uniformly on $[0, 2\pi)$. Particle speeds are sampled via rejection sampling using

$$g(q) = q^2(1-q^2)^{\frac{7}{2}} \tag{14}$$

against the local escape velocity

$$v_e = \sqrt{\frac{2GM}{\sqrt{r^2+a^2}}} \tag{15}$$

with isotropic velocity directions. This scenario tests tree accuracy and long-term stability in a compact three-dimensional distribution. The Plummer softening parameter ε in the force model is conceptually related to the Plummer scale length a in the initial conditions, though the two are independently configurable. The key sweep variables are θ, ε, and Δt; the primary limitation is that the spherical geometry does not emphasize disk morphology such as spirals and bars.

2.9.3 Scenario C: rotating exponential disk (galaxy-like morphology test)

Scenario C generates a rotating disk with $N \in [10^4, 10^5]$ to evaluate long-term evolution and visually interpretable galactic dynamics. The surface density profile follows an exponential distribution (Freeman, 1970), with particle radii drawn from an exponential distribution (rate parameter 0.08) and clamped to a maximum of 50 units, and azimuthal angles drawn uniformly. Vertical height is drawn from a normal distribution $\mathcal{N}(0, 0.3)$ scaled by $\frac{1}{1+0.5r}$ to produce a thinner disk at larger radii. Particle masses are drawn uniformly from $[0.5, 2.0]$.

Circular velocities are assigned using an approximate enclosed-mass estimate. For particles with $r > 0.1$, the tangential velocity is

$$v = 0.5\sqrt{\frac{M_{\text{enclosed}}}{r}} \tag{16}$$

with the velocity directed tangentially. This simplified dynamical setup is not a full multi-component Milky Way model: the enclosed-mass estimate is approximate, and no bulge or halo component is included. Nevertheless, the disk geometry provides a morphologically rich test case in which the formation and persistence of spiral structure, bars, and other large-scale features serve as qualitative validation of the solver's long-term behavior. The key sweep variables are disk scale length, thickness, velocity dispersion, θ, and Δt.

2.9.4 Sampling and robustness across seeds

Because initial conditions are stochastic, robustness is assessed by repeating runs with different seeds (`--seed 1`, `--seed 2`, and so on) and comparing diagnostics and timing. Initial condition generation uses `std::mt19937` seeded by the `--seed` parameter (default 42), ensuring deter-

ministic reproduction of particle distributions and velocities. Runs are considered valid if they complete without NaN or overflow values and produce consistent parameter logs. Deliberately unstable configurations (such as excessively large Δt) are retained as documented failures for robustness reporting rather than silently excluded.

Placeholder: Three-panel figure showing initial particle distributions. Left: Scenario A (two-body orbit, two particles with velocity vectors). Center: Scenario B (Plummer sphere, spherically symmetric particle cloud). Right: Scenario C (exponential disk, face-on view showing disk structure).

Figure 5: Initial particle distributions for the three benchmark scenarios. (a) Scenario A: two-body orbit. (b) Scenario B: Plummer sphere with $N = 10000$. (c) Scenario C: exponential disk with $N = 50000$.

The evaluation protocol systematically exercises these scenarios across the parameter space defined above.

2.10 Evaluation Protocol: Baselines, Metrics, and Parameter Sweeps

2.10.1 Baselines

Two baselines are used. The first is a direct $O(N^2)$ summation, which serves as a reference computation path for small N. Potential energy is computed by direct pair summation only when $N \leq 5000$ due to its $O(N^2)$ cost. The second baseline is the forward Euler integrator (`--integrator euler`), which provides a numerical stability reference against which the leapfrog scheme is compared.

2.10.2 Primary metrics

Runtime per timestep (measured in milliseconds per step using `std::chrono::high_resolution_clock`) is decomposed into three components: tree build time (GPU LBVH construction or CPU octree construction plus upload), force evaluation time (GPU BVH traversal or CPU Barnes–Hut walk), and integration time (kick and drift dispatches, including any CPU mirror loops on fallback paths). This decomposition identifies which phase dominates at each particle count and reveals the tree-build fraction, which is a key indicator of the overhead introduced by per-step hierarchy construction.

Scaling with particle count is characterized by measuring total and per-component runtimes across a range of N values for both hierarchical and direct modes, producing empirical scaling curves that are compared against the expected $O(N \log N)$ and $O(N^2)$ asymptotic behavior. Long-term stability is assessed through energy drift, defined as $\Delta E(t) = |E(t) - E(0)|/|E(0)|$, reported only for $N \leq 5000$ where potential energy is computed via direct pair summation. For larger N, stability is assessed through kinetic energy trends and momentum diagnostics.

2.10.3 Secondary metrics

Linear momentum magnitude $\|P(t)\| = \|\sum_i m_i v_i\|$ is computed in double precision and is expected to remain near zero for symmetric initial conditions (Plummer sphere) while reflecting the net angular momentum of the disk scenario. Qualitative morphology in disk runs (persistence of spiral arms, bar formation, and overall structural evolution) is reported descriptively as a complement to quantitative diagnostics, providing a visual sanity check on solver behavior over long integration times.

2.10.4 Parameter sweeps

To characterize accuracy–performance trade-offs and stability regimes, controlled parameter sweeps are performed via the command-line interface. The opening angle θ is swept over $\{0.3, 0.5, 0.7, 1.0\}$, the timestep Δt is varied across a scenario-dependent stable range, and the softening parameter ε is tested across representative values. Each sweep also compares the Euler and leapfrog integrators and the CPU octree versus GPU LBVH construction paths. Results are compared using exported CSV logs containing per-step diagnostics and timing.

2.10.5 Error analysis procedure

When instability or anomalous drift occurs, runs are inspected for correlations with dense versus diffuse particle regions, large θ, large Δt, small ε, and platform or backend differences. Failures (NaN values, overflow, and extreme velocities) are detected explicitly and logged rather than silently suppressed.

2.10.6 Comparative positioning

To situate the performance of this implementation within the broader landscape of GPU N-body codes, key metrics are mapped to those reported in existing literature. Burtscher and Pingali report throughput for a CUDA Barnes–Hut implementation on NVIDIA hardware (Burtscher & Pingali, 2011), and Gaburov, Bédorf, and Portegies Zwart report CUDA tree-code performance including tree-build fractions at varying N (Gaburov et al., 2010). Nyland, Harris, and Prins provide reference throughput figures for GPU N-body methods in the CUDA ecosystem (Nyland et al., 2009). While direct comparison is limited by differences in hardware generation, precision (64-bit versus 32-bit), tree topology (octree versus BVH), and opening criteria, the

metrics collected in this work (throughput at given N, tree-build fraction of total step time, and crossover N between direct and hierarchical methods) are chosen to enable meaningful comparison with these published results.

Beyond the quantitative metrics, the methodology incorporates several validation strategies to ensure correctness and robustness.

2.11 Validation and Robustness

The validation strategy rests on three pillars: theoretical grounding, empirical checks, and explicit acknowledgment of practical constraints. On the theoretical side, the Barnes–Hut hierarchical approximation with tunable opening angle θ follows Barnes and Hut (Barnes & Hut, 1986), the leapfrog integration scheme is a standard choice for long-horizon gravitational dynamics due to its symplectic stability properties (Springel, 2005), the GPU hierarchy construction follows the widely used parallel LBVH method of Karras (Karras, 2012), and the GPU traversal methodology follows established considerations for irregular tree algorithms on SIMD-style hardware (Burtscher & Pingali, 2011; Nyland et al., 2009).

Empirical validation is provided at multiple levels. Scenario A (two-body orbit) offers a controlled sanity check for orbit stability and integrator correctness, where deviations from the analytic circular orbit can be directly measured. Deterministic seeds and complete parameter logging ensure that any run can be exactly reproduced. Fixed step counts (`--steps`) provide consistent comparison across runs, and instability events are recorded rather than filtered. The availability of both CPU octree and GPU BVH execution paths allows cross-validation of force evaluation results for the same initial conditions.

Practical constraints are explicitly acknowledged. Interactive runs couple stepping to the render loop, while headless mode prioritizes throughput. Particle count is adjustable from 2 to 100,000. Potential energy tracking is intentionally limited to $N \leq 5000$ to avoid prohibitive $O(N^2)$ overhead; for larger N, stability is assessed through kinetic energy and momentum diagnostics combined with qualitative morphological assessment. These precision and buffer constraints connect directly to the WebGPU platform limitations described in the platform justification section, where 32-bit arithmetic and device variability were identified as fundamental characteristics of the target environment.

2.12 Data Ethics, Security, and Integrity

No personal or sensitive data are collected. All computation runs locally (native) or within the user's browser sandbox (Emscripten). Diagnostic logs and trajectories are exported only when explicitly requested (`--export`). Each CSV export is linked to the complete set of runtime

parameters (scenario, N, Δt, θ, ε, seed, integrator, steps). The primary practical risk is high GPU load; mitigation is provided through adjustable N and configurable step limits.

2.13 Reproducibility and Traceability

Reproducibility of build and configuration is ensured through several mechanisms. Initial condition generation uses `std::mt19937` with the seed recorded at startup (default 42), producing deterministic particle distributions and velocities for any given parameter set. All third-party dependency versions are pinned through CMake FetchContent, and the complete set of CLI parameters is logged at startup via spdlog alongside the GPU adapter name and selected WebGPU backend, creating a full record of the execution environment for each run.

Output reproducibility and cross-environment comparison are supported by the CSV export format, which records per-step diagnostics and timing: step number, simulation time, kinetic energy, potential energy (when $N \leq 5000$), total energy, energy drift, momentum components (p_x, p_y, p_z), tree-build time, force-evaluation time, and integration time. The dual build targets (native and browser) are produced from the same source code, enabling direct cross-environment comparison of both numerical results and performance characteristics under identical simulation parameters.

References

Aarseth, S., Henon, M., & Wielen, R. (1974). A Comparison of Numerical Methods for the Study of Star Cluster Dynamics. *\aap*, *37*(1), 183–187.

Barnes, J., & Hut, P. (1986). A hierarchical O(N log N) force-calculation algorithm. *Nature*, *324*(6096), 446–449. https://doi.org/10.1038/324446a0

Batcher, K. E. (1968). Sorting Networks and Their Applications. *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference*, 307–314. https://doi.org/10.1145/1468075.1468121

Binney, J., & Tremaine, S. (2008). *Galactic Dynamics: Second Edition* (REV - Revised, 2). Princeton University Press. http://www.jstor.org/stable/j.ctvc778ff

Burtscher, M., & Pingali, K. (2011). An Efficient CUDA Implementation of the Tree-Based Barnes Hut n-Body Algorithm. *GPU Computing Gems Emerald Edition*, . https://doi.org/10.1016/B978-0-12-384988-5.00006-1

Congote, J., Segura, A., Kabongo, L., Moreno, A., Posada, J., & Ruiz, O. (2011). Interactive visualization of volumetric data with WebGL in real-time. *Proceedings of the 16th International Conference on 3d Web Technology*, 137–146. https://doi.org/10.1145/2010425.2010449

Fluke, C. J., Barnes, D. G., Barsdell, B. R., & Hassan, A. H. (2011). Astrophysical Supercomputing with GPUs: Critical Decisions for Early Adopters. *Publications of the Astronomical Society of Australia*, *28*(1), 15–27. https://doi.org/10.1071/as10019

Freeman, K. C. (1970). On the Disks of Spiral and S0 Galaxies. *The Astrophysical Journal*, *160*, 811. https://doi.org/10.1086/150474

Gaburov, E., Bédorf, J., & Portegies Zwart, S. (2010). Gravitational tree-code on graphics processing units: Implementation in CUDA. *Procedia Computer Science*, *324*, 1119–1127. https://doi.org/10.1016/j.procs.2010.04.124

GPU for the Web Working Group. (2026,). *WebGPU (GPU for the Web) Specification*.

Karras, T. (2012). Maximizing parallelism in the construction of BVHs, octrees, and k-d trees. *Proceedings of the Fourth ACM SIGGRAPH / Eurographics Conference on High-Performance Graphics*, 33–37.

Khronos Group. (2024,). *WebGPU Specification*.

Kreyszig, E. (2011). *Advanced Engineering Mathematics* (10th ed.). Wiley.

LF, G., & Rokhlin, V. (2001). A Fast Algorithm for Particle Simulation. *Journal of Computational Physics*, *73*, 325–348. https://doi.org/10.1016/0021-9991(87)90140-9

Morton, G. M. (1966). *A Computer Oriented Geodetic Data Base and a New Technique in File Sequencing*.

Nyland, L., Harris, M., & Prins, J. (2009). Fast N-body simulation with CUDA. *GPU Gem, Vol. 3*, 677–695.

Owens, J. D., Luebke, D., Govindaraju, N., Harris, M., Krüger, J., Lefohn, A. E., & Purcell, T. J. (2007). A Survey of General-Purpose Computation on Graphics Hardware. *Computer Graphics Forum*, *26*(1), 80–113. https://doi.org/https://doi.org/10.1111/j.1467-8659.2007.01012.x

Owens, J., Houston, M., Luebke, D., Green, S., Stone, J., & Phillips, J. (2008). GPU computing. *Proceedings of the IEEE*, *96*, 879–899. https://doi.org/10.1109/JPROC.2008.917757

Plummer, H. C. (1911). On the Problem of Distribution in Globular Star Clusters. *Monthly Notices of the Royal Astronomical Society*, *71*(5), 460–470. https://doi.org/10.1093/mnras/71.5.460

Salmon, J. K., & Warren, M. S. (1994). Skeletons from the Treecode Closet. *Journal of Computational Physics*, *111*(1), 136–155. https://doi.org/https://doi.org/10.1006/jcph.1994.1050

Sengupta, S., Wu, N., Varvello, M., Jana, K., Chen, S., & Han, B. (2025). From WebGL to WebGPU: A Reality Check of Browser-Based GPU Acceleration. *Proceedings of the 2025 ACM Internet Measurement Conference*, 1018–1024. https://doi.org/10.1145/3730567.3764504

Springel, V. (2005). The cosmological simulation code gadget-2. *Monthly Notices of the Royal Astronomical Society*, *364*(4), 1105–1134. https://doi.org/10.1111/j.1365-2966.2005.09655.x

Sung, N.-J., Ma, J., Kim, T., Choi, Y.-j., Choi, M.-H., & Hong, M. (2025,). *Real-Time Cloth Simulation Using WebGPU: Evaluating Limits of High-Resolution*. https://arxiv.org/abs/2507.11794

Sutter, H. (2005). The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software. *Dr. Dobb's Journal*, *30*(3), 202–210. http://www.gotw.ca/publications/concurrency-ddj.htm

Usta, Z. (2024). WEBGPU: A NEW GRAPHIC API FOR 3D WEBGIS APPLICATIONS. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, 377–382. https://doi.org/10.5194/isprs-archives-XLVIII-4-W9-2024-377-2024

Verlet, L. (1967). Computer ``Experiments" on Classical Fluids. I. Thermodynamical Properties of Lennard-Jones Molecules. *Physical Review*, *159*(1), 98–103. https://doi.org/10.1103/PhysRev.159.98

Wang, Q. (2021). A hybrid Fast Multipole Method for cosmological N-body simulations. *Research in Astronomy and Astrophysics*, *21*(1), 3. https://doi.org/10.1088/1674-4527/21/1/3

Zwart, S. F. P., Belleman, R. G., & Geldof, P. M. (2007). High-performance direct gravitational N-body simulations on graphics processing units. *New Astronomy*, *12*(8), 641–650. https://doi.org/https://doi.org/10.1016/j.newast.2007.05.004